

Experiment No.	5
Experiment Title	Decision Tree and Random Forest: A Comparative Classification Study ¹
Student Name	Raghu
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 23, 2026

1 Objective

- To implement a Decision Tree classifier.
- To extend the Decision Tree into a Random Forest ensemble model.
- To study the impact of hyperparameters on overfitting and generalization.
- To select optimal hyperparameters using 5-Fold Cross-Validation.
- To compare single-tree and ensemble-tree models.

2 Problem Statement

Given 30 numerical attributes computed from digitized images of a breast mass fine-needle aspirate, predict whether the mass is *Malignant* (1) or *Benign* (0). The input is a 569×30 feature matrix; the output is a binary class prediction. The objective is to build, tune, and compare a single Decision Tree against a Random Forest ensemble on this task.

3 Python Libraries Used

```

1 numpy, pandas, matplotlib, seaborn
2 scikit-learn:
3   - sklearn.datasets (load_breast_cancer)
4   - sklearn.model_selection (train_test_split, GridSearchCV,
5                               StratifiedKFold, cross_val_score)
6   - sklearn.tree (DecisionTreeClassifier, plot_tree)
7   - sklearn.ensemble (RandomForestClassifier)
8   - sklearn.metrics (accuracy_score, precision_score,
9                       recall_score,
10                      f1_score, roc_auc_score, roc_curve,
11                      confusion_matrix, ConfusionMatrixDisplay,
12                      classification_report)
13 json, time, pathlib, warnings

```

4 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (loaded via <code>sklearn.datasets.load_breast_cancer</code> , identical to the archived UCI copy)
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard error, and worst value of 10 nuclear morphology measurements)
Number of Classes	2 (Malignant = 1, Benign = 0)
Missing Values	0
Duplicate Rows	0
Class Balance	357 Benign, 212 Malignant
Train-Test Split	80% / 20%, stratified (455 train / 114 test)

5 Exploratory Data Analysis

Class Distribution

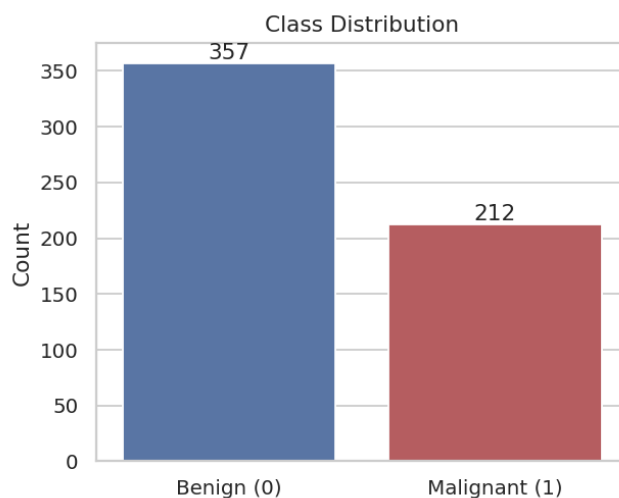


Figure 1: Distribution of Benign vs. Malignant diagnoses (357 Benign, 212 Malignant).

Inference: The dataset is moderately imbalanced (roughly 63% Benign, 37% Malignant), similar in degree to the Spambase imbalance seen in Experiment 4. Stratified splitting was used to preserve this ratio across train and test sets, and precision/recall/F1 are reported alongside accuracy since the clinical cost of a false negative (missed malignancy) is far higher than a false positive.

Feature Correlation Analysis

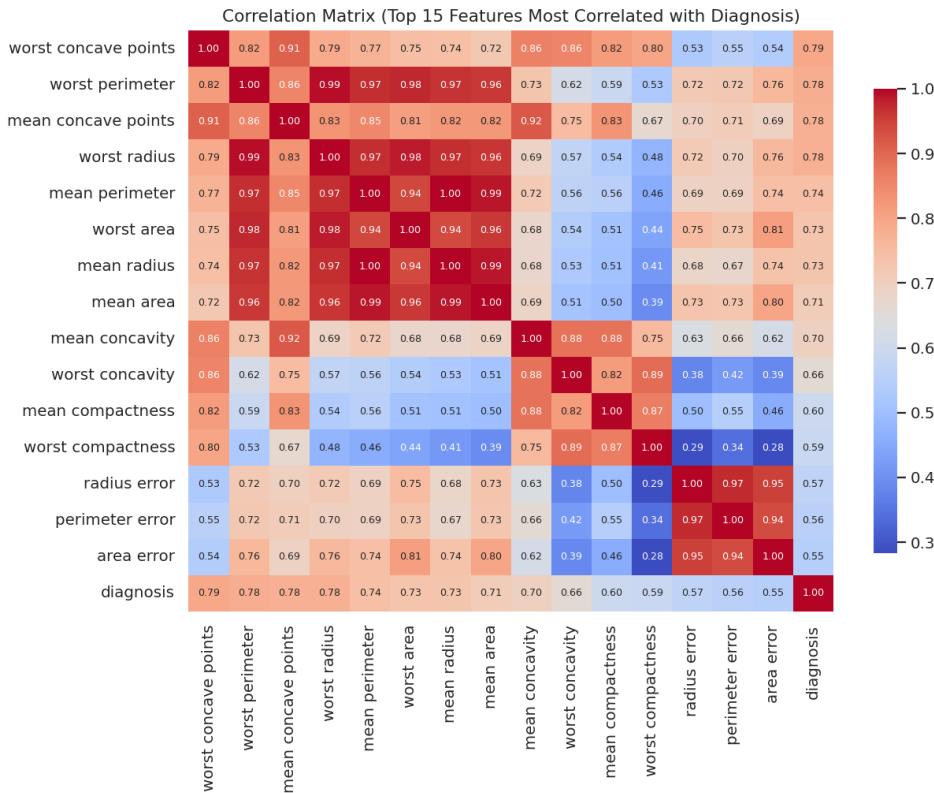


Figure 2: Correlation heatmap of the 15 features most correlated with diagnosis.

Inference: Size- and shape-related features (worst/mean radius, perimeter, area, and concave points) show the strongest correlation with malignancy, consistent with the clinical intuition that larger, more irregularly-shaped nuclei indicate cancerous cells. Several of these features are also highly correlated with each other (e.g., radius, perimeter, and area are geometrically dependent), which is well tolerated by tree-based models since they select one informative feature per split rather than requiring linear independence.

Feature Distribution by Class

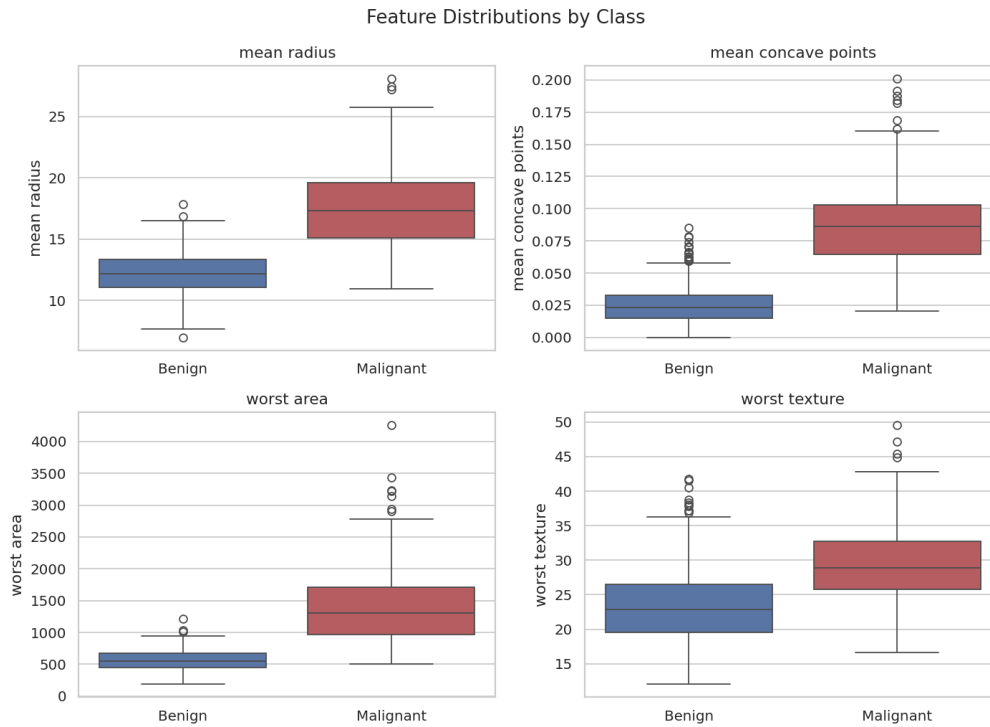


Figure 3: Distributions of four representative features, split by class.

Inference: Malignant samples show clearly higher medians and wider spread for mean radius, mean concave points, and worst area compared to Benign samples, with limited overlap between the two class distributions. This strong per-feature separability explains why even a shallow Decision Tree can achieve high accuracy on this dataset.

6 Data Preprocessing

- **Label encoding:** Target relabeled so Malignant = 1 (positive class), Benign = 0.
- **Missing value handling:** None required — zero missing entries in all 30 features.
- **Duplicate handling:** None required — zero duplicate rows found.
- **Feature scaling:** Not applied. Decision Trees and Random Forests split on individual feature thresholds and are invariant to monotonic feature scaling, unlike the distance/margin-based models used in Experiment 4.
- **Train-test split:** 80%/20% stratified split (`random_state=42`) to preserve class ratio in both partitions.

7 Machine Learning Algorithm

Decision Tree Classifier

Working principle: A Decision Tree recursively partitions the feature space by choosing, at each node, the feature and threshold that most reduces class impurity — measured by the Gini Index or Entropy:

$$\text{Gini} = 1 - \sum_i p_i^2 \quad \text{Entropy} = - \sum_i p_i \log_2 p_i$$

Splitting continues until a stopping criterion (max depth, minimum samples per node) is reached, producing a set of axis-aligned decision rules.

Advantages: Highly interpretable (rules can be read directly off the tree), requires no feature scaling, and naturally handles non-linear decision boundaries and feature interactions.

Limitations: Prone to high variance — small changes in training data can produce very different trees — and deep, unconstrained trees overfit readily.

Random Forest Classifier

Working principle: Random Forest trains many decision trees, each on a bootstrap-resampled subset of the training data and considering only a random subset of features at each split, then aggregates their predictions by majority vote:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_n(x)\}$$

This decorrelates the individual trees so their errors partially cancel out when averaged.

Advantages: Substantially reduces variance and overfitting relative to a single tree, typically improves generalization, and provides a built-in feature-importance ranking.

Limitations: Less interpretable than a single tree, more computationally expensive to train and tune, and can still overfit with too few estimators or excessive per-tree depth.

8 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Claude Code sandbox (headless)
Operating System	Ubuntu 24.04 (Linux container)
Hardware	CPU-only container

9 Hyperparameter Selection

Parameter	Search Space
DT: criterion	gini, entropy
DT: max_depth	3, 5, 7, 10, None
DT: min_samples_split	2, 5, 10
DT: min_samples_leaf	1, 2, 4
RF: n_estimators	100, 200, 300
RF: max_depth	5, 10, None
RF: max_features	sqrt, log2
RF: bootstrap	True, False

Both models were tuned with exhaustive **Grid Search** (5-fold cross-validation, scoring = accuracy): 90 combinations $\times$ 5 folds = 450 fits for the Decision Tree, and 36 combinations $\times$ 5 folds = 180 fits for Random Forest.

10 Results

Table 1: Decision Tree Hyperparameter Evaluation (Top Configurations)

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
Entropy	7	94.73	0.9297
Entropy	10	94.73	0.9297
Entropy	None	94.73	0.9297
Gini	7	94.29	0.9227
Gini	None	93.85	0.9167
Entropy	5	93.85	0.9176
Gini	10	93.85	0.9167
Gini	5	93.41	0.9097
Gini	3	93.19	0.9063
Entropy	3	92.09	0.8944

Selected best configuration: criterion = entropy, max_depth = 7, min_samples_split = 5, min_samples_leaf = 2 (Avg CV Accuracy = 94.73%). Depths 7 and 10 (and unlimited depth) tie in this table because min_samples_leaf/min_samples_split already constrain effective tree growth beyond depth ≈ 7 for this dataset.

Table 2: Random Forest Hyperparameter Evaluation (Top Configurations)

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
200	10	log2	97.14	0.9618
100	10	log2	96.92	0.9592
100	None	log2	96.92	0.9592
200	None	log2	96.70	0.9555
100	None	sqrt	96.48	0.9528
300	None	log2	96.48	0.9525
300	10	log2	96.48	0.9525
100	10	sqrt	96.48	0.9528
300	10	sqrt	96.26	0.9494
200	10	sqrt	96.26	0.9491

Selected best configuration: n_estimators = 200, max_depth = 10, max_features = log2, bootstrap = False (Avg CV Accuracy = 97.14%).

Table 3: 5-Fold Cross-Validation Accuracy Comparison (Tuned Models, Training Set)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.9011	0.9670	0.9231	0.9121	0.9451	0.9297 (± 0.0237)
Random Forest	0.9560	0.9890	0.9451	0.9670	0.9670	0.9648 (± 0.0146)

Held-Out Test Set Performance (Tuned Models)

Metric	Decision Tree	Random Forest
Accuracy	0.9561	0.9649
Precision	1.0000	1.0000
Recall	0.8810	0.9048
F1 Score	0.9367	0.9500
ROC-AUC	0.9396	0.9940
Training Time (s)	0.006	0.347

11 Comparative Analysis

Criterion	Decision Tree	Random Forest
Accuracy (test set)	0.9561	0.9649
Model Complexity	Low	High
Training Time	Low (0.006 s)	Higher (0.347 s)
Interpretability	High	Low

Observations

- **How does tree depth affect overfitting?** Figure 4 shows training accuracy climbing to a perfect 1.0 by depth ≈ 8 , while test accuracy peaks around depth 7 (93.9%) and then plateaus, even dips, as depth grows further. The widening gap between the two curves past depth ~ 5 is a textbook overfitting signature — the unconstrained tree keeps memorizing training noise without any corresponding test-set benefit.
- **Which hyperparameter had the greatest impact?** For the Decision Tree, `max_depth` had by far the largest effect — accuracy rose from roughly 87.7% at depth 1 to its peak by depth 7, dwarfing the smaller gains from tuning `criterion`, `min_samples_split`, or `min_samples_leaf`. For Random Forest, `max_features=log2` consistently outperformed `sqrt` across every (`n_estimators`, `max_depth`) pair in Table 2, making it the most influential single knob, with `n_estimators` contributing smaller, diminishing-return gains beyond 100 trees.
- **How does Random Forest improve generalization?** By averaging predictions across many trees, each trained on a bootstrapped sample with a random feature subset at every split, individual trees' errors are decorrelated and tend to cancel out on aggregation. This is visible directly in Table 3: Random Forest's cross-validation standard deviation (0.0146) is essentially half the Decision Tree's (0.0237), and its ROC-AUC (0.9940) is far higher and more robust than the single tree's (0.9396), despite both achieving perfect test precision.
- **Did ensemble learning always improve performance?** Here, yes on every metric — Random Forest beat the Decision Tree on accuracy, recall, F1, ROC-AUC, and CV stability. This is not guaranteed in general: ensembling mainly helps when individual trees are high-variance and only weakly correlated with each other; on very small or very low-noise datasets, a well-tuned single tree can sometimes match a forest at a fraction of the training cost, and forests offer no benefit if all trees converge to nearly the same splits (e.g., when one feature dominates the correlation structure, as seen here with the strong radius/perimeter/area correlations in Figure 2).

12 Result Visualization

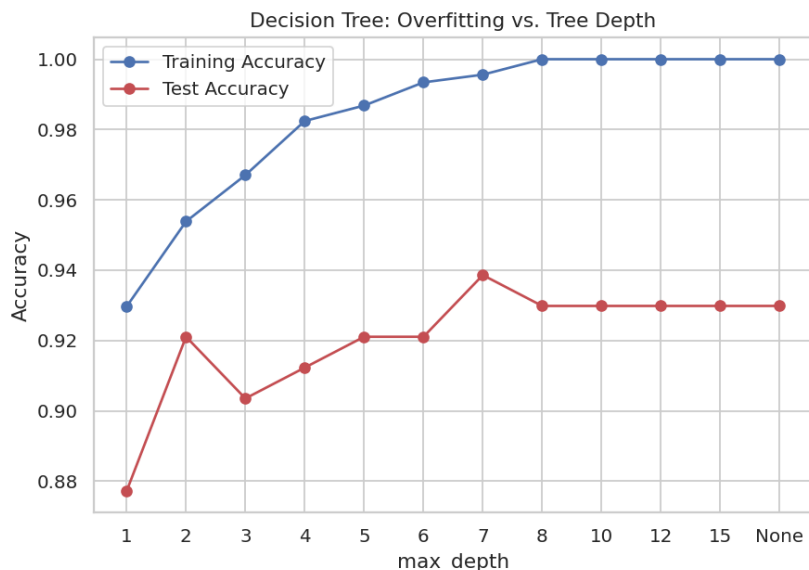


Figure 4: Training vs. test accuracy across increasing `max_depth` (Decision Tree, default other params).

Inference: Training accuracy reaches 100% by depth 8 while test accuracy plateaus near 93%, visually confirming the overfitting behavior discussed above. The optimal generalization point (highest test accuracy without a growing gap) sits around depth 6–7, matching the value chosen by grid search.

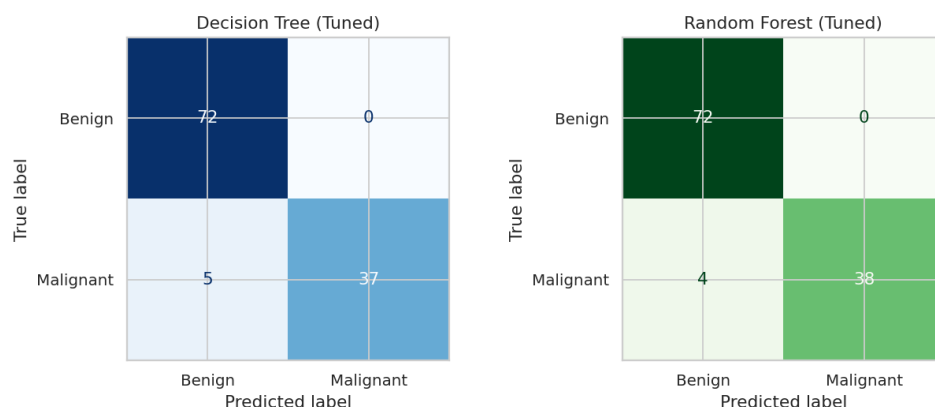


Figure 5: Confusion matrices for tuned Decision Tree (left) and tuned Random Forest (right) on the test set.

Inference: Both models achieve perfect precision (zero false positives — no benign case misclassified as malignant), but the Decision Tree misses more malignant cases (5 false negatives) than Random Forest (4 false negatives) out of 114 test samples. In a clinical screening context, minimizing false negatives is the priority, so Random Forest's slightly better recall (0.9048 vs. 0.8810) is a meaningful practical advantage.

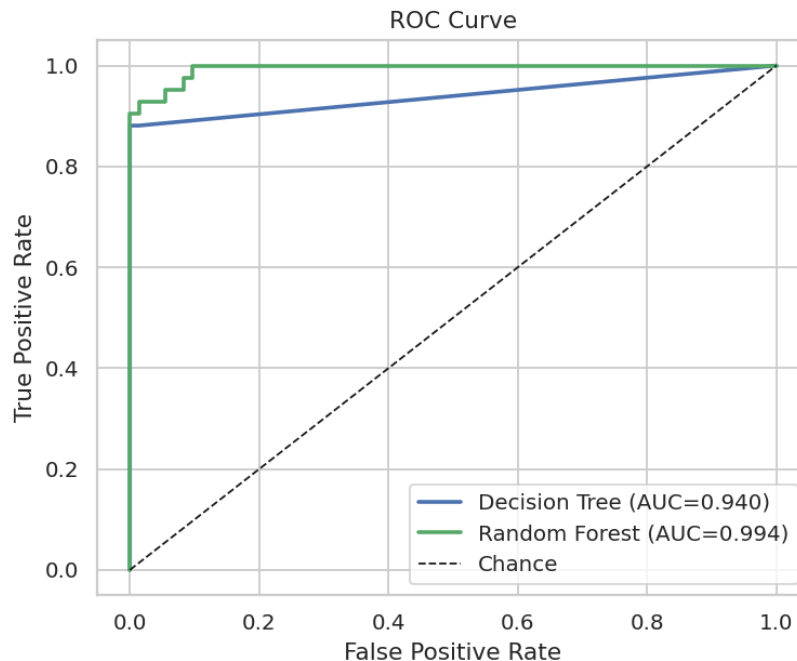


Figure 6: ROC curves for tuned Decision Tree and tuned Random Forest.

Inference: Random Forest’s ROC curve dominates the Decision Tree’s across almost the entire false-positive-rate range, with AUC of 0.9940 vs. 0.9396. Unlike Experiment 4’s SVM/LR comparison where AUCs were nearly identical, here the ensemble shows a clear, substantial ranking advantage — a single tree’s piecewise-constant probability estimates are far coarser than a forest’s averaged, smoothly-varying ones.

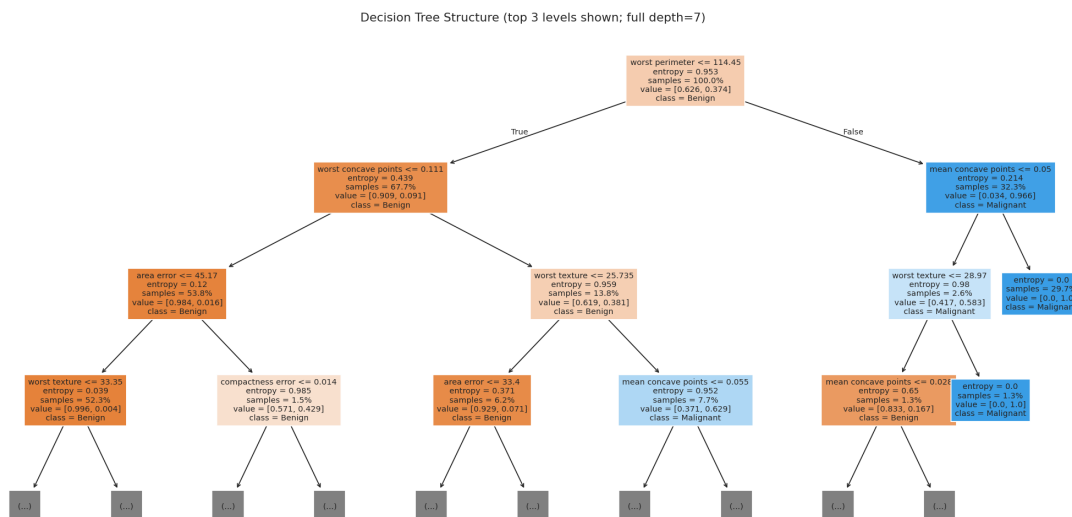


Figure 7: Learned Decision Tree structure (top 3 levels shown; full tuned tree reaches depth 7).

Inference: The root split is on **worst perimeter**, immediately separating a large, mostly-pure Benign leaf from a mixed subgroup that is further split on **worst concave**

`worst perimeter` and `worst texture` — consistent with the correlation analysis identifying these as the most diagnostically informative measurements. The tree reaches high purity within just 2–3 levels for most samples, explaining why shallow depths already perform competitively.

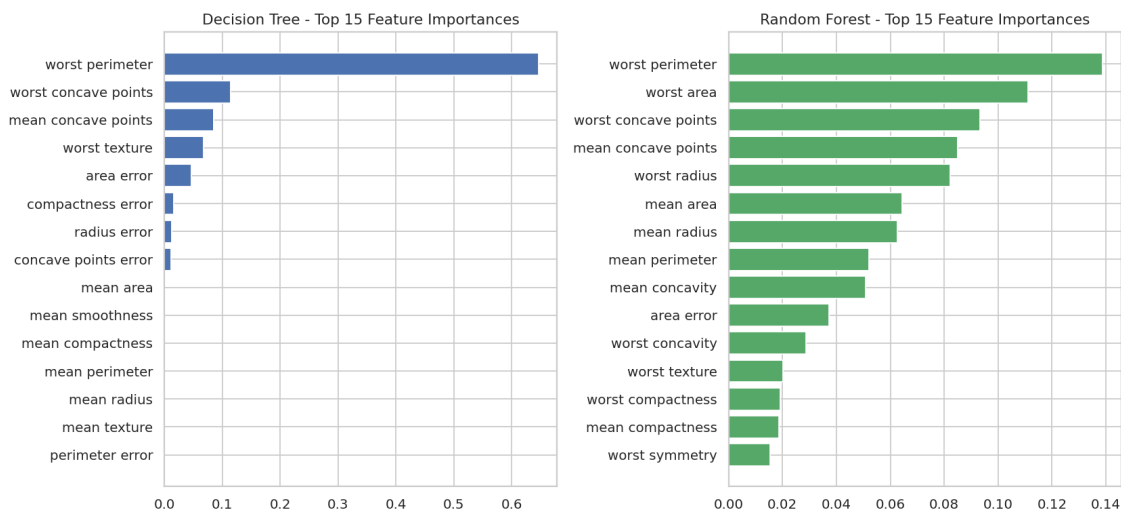


Figure 8: Top 15 feature importances for Decision Tree (left) and Random Forest (right).

Inference: Both models agree that `worst perimeter`/`worst area`/`worst concave points` are the most important predictors, but the Random Forest importance distribution is noticeably flatter — it distributes credit across more features (`worst radius`, `mean concave points`, `worst concavity`) rather than concentrating almost all importance on the top 1–2 splits as the single Decision Tree does. This spreading-out is a direct consequence of random feature subsetting at each split, and is part of why the ensemble generalizes more robustly.

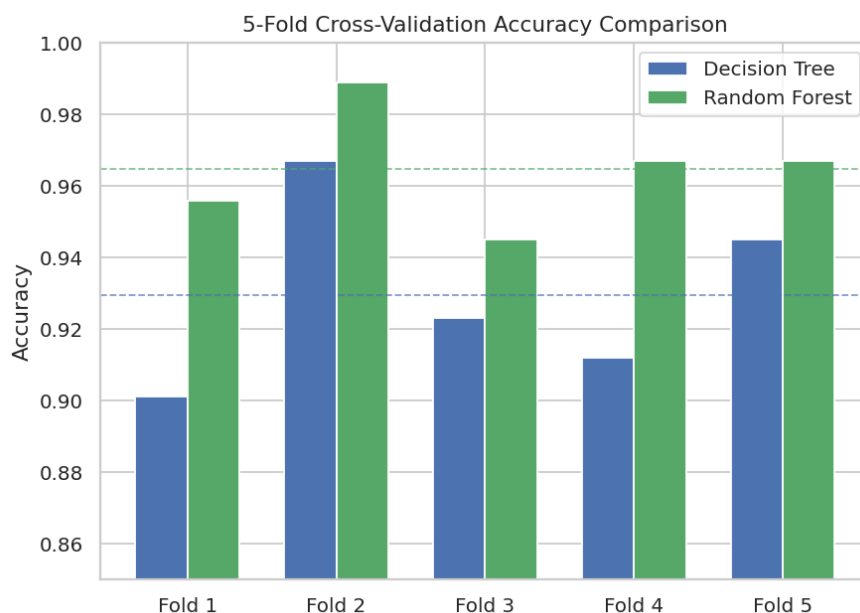


Figure 9: Per-fold 5-fold cross-validation accuracy comparison.

Inference: Random Forest outperforms the Decision Tree on every single fold, not just on average, with a visibly tighter spread (0.945–0.989) compared to the Decision Tree’s wider swing (0.901–0.967). This fold-by-fold consistency is stronger evidence of reduced variance than the average alone would suggest.

13 Discussion

Random Forest outperformed the single Decision Tree on every evaluation axis in this experiment — accuracy, recall, F1, ROC-AUC, and cross-validation stability — at the cost of roughly $55\times$ longer training time (0.347s vs. 0.006s) and substantially reduced interpretability. On a dataset this small (569 samples, 30 features), that training-time cost is negligible in absolute terms, so Random Forest is the clear practical choice here; the trade-off would matter more at production scale with millions of rows or strict latency budgets. The clearest single lesson from the overfitting curve (Figure 4) is that Decision Trees need explicit depth/leaf constraints to generalize — an unconstrained tree reaches 100% training accuracy while gaining nothing on the test set past depth ~ 7 . A limitation of this study is that grid search optimized for overall accuracy rather than recall specifically; given the clinical cost asymmetry between false negatives and false positives, a metric like F2-score (which weights recall higher) or a cost-sensitive threshold might be more appropriate for a real diagnostic-support tool. Future work could explore boosting methods (e.g., Gradient Boosting, XGBoost) as a further comparison point, and SHAP-based feature attribution to make the Random Forest’s decisions more interpretable for clinical use.

14 Conclusion

Decision Tree and Random Forest models were implemented and evaluated using 5-fold cross-validation on the WDBC dataset. Hyperparameters were selected based on average cross-validation accuracy, yielding an entropy-criterion tree of depth 7 (94.73% CV accuracy) and a 200-tree Random Forest with `max_depth=10`, `max_features=log2` (97.14% CV accuracy). On the held-out test set, Random Forest achieved higher accuracy (96.49% vs. 95.61%), recall (0.905 vs. 0.881), F1 (0.950 vs. 0.937), and substantially higher ROC-AUC (0.994 vs. 0.940) than the single Decision Tree, while also showing tighter, more consistent cross-validation performance. These results confirm that Random Forest’s bagging and random feature selection reduce variance and improve generalization compared to a single Decision Tree, at a modest, acceptable training-time cost for a dataset of this size.

15 Learning Outcomes

- Understood how impurity measures (Gini Index and Entropy) drive recursive splitting in Decision Trees.
- Applied hyperparameter tuning via Grid Search with 5-fold cross-validation to both a single tree and an ensemble model.
- Observed empirically how tree depth controls the overfitting/underfitting trade-off.

- Evaluated classification models using accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC.
- Interpreted experimental results to explain how bagging and random feature selection reduce variance in ensemble learning.

16 References

- 1 F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- 2 Scikit-learn Developers, “Decision Trees,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
- 3 Scikit-learn Developers, “Ensemble methods: Random Forests,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html#random-forests>
- 4 W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” *IS&T/SPIE International Symposium on Electronic Imaging*, 1993.
- 5 “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>